Ticket Booking — Performance Benchmark Interview Guide

Project: CineBook / Ticket Booking
Prepared: 16 October 2026
Purpose: Tài liệu ôn phỏng vấn về cách đo và giải thích các tối ưu performance trong project.

> **Important / Quan trọng:** Các số liệu là **local microbenchmark** trên dữ liệu synthetic trong PostgreSQL cô lập. Đây không phải production load test, production SLO hoặc số liệu từ dữ liệu người dùng thật.

1. One-minute answer / Câu trả lời một phút

English

> I built a full-stack movie ticket booking platform with NestJS, React, PostgreSQL, Prisma, and Redis. For performance work, I focused on database and ORM behavior rather than browser rendering. I created an isolated Docker PostgreSQL database with the same Prisma migrations, generated synthetic data, ran warm-up iterations, and compared the baseline and optimized approaches under the same conditions. I measured median and P95 latency, counted Prisma query events, and used `EXPLAIN ANALYZE BUFFERS` to inspect PostgreSQL query plans. With 100,000 synthetic bookings, deep offset pagination at offset 90,000 had a median latency of 7.38 ms, while cursor pagination with a composite index measured 0.26 ms. I also reduced seat price updates from 120 individual update statements to 3 grouped updates.

Tiếng Việt

> Tôi xây dựng hệ thống đặt vé xem phim full-stack với NestJS, React, PostgreSQL, Prisma và Redis. Khi tối ưu performance, tôi tập trung đo tầng database và ORM thay vì đo rendering của browser. Tôi tạo một PostgreSQL database riêng bằng Docker với cùng bộ Prisma migration, sinh dữ liệu synthetic, chạy warm-up rồi so sánh cách cũ và cách tối ưu trong cùng điều kiện. Tôi đo median và P95 latency, đếm số Prisma query event và dùng `EXPLAIN ANALYZE BUFFERS` để kiểm tra query plan của PostgreSQL. Với 100.000 booking synthetic, offset pagination ở offset 90.000 có median latency 7.38 ms, trong khi cursor pagination kết hợp composite index là 0.26 ms. Tôi cũng giảm việc cập nhật giá ghế từ 120 câu lệnh update riêng lẻ xuống còn 3 câu lệnh update theo nhóm.

2. Project context / Bối cảnh project

English

- Backend: NestJS modular monolith, Prisma and PostgreSQL.
- Frontend: React/Vite.
- Redis: distributed seat locks with TTL.
- Flow: showtime discovery, seat holding, booking, SePay QR payment, webhook reconciliation and ticket issuing.

Tiếng Việt

- Backend: NestJS modular monolith, Prisma và PostgreSQL.
- Frontend: React/Vite.
- Redis: distributed lock cho ghế với TTL.
- Flow: xem suất chiếu, giữ ghế, booking, thanh toán QR SePay, webhook và phát hành vé.

- Invariant: one seat for one showtime must not be sold twice.
- Invariant: một ghế trong một suất không được bán trùng.

3. Benchmark environment / Môi trường đo

Item	Value
Database	PostgreSQL 17 in isolated Docker Compose
Redis	Redis 7, benchmark port 56381
PostgreSQL port	55434
Migrations	Same Prisma migrations as the project
External services	No SePay, TMDB, SMTP or production frontend
Script	`scripts/benchmark-ticket-booking.mjs`
Runs	2 warm-ups + 10 measured iterations
Reported metrics	Median, P95, Prisma query count, result size and query plan

Why no frontend? / Vì sao không mở frontend?

English: The goal was to isolate database and ORM performance. Including browser rendering and network latency would add unrelated variables.

Tiếng Việt: Mục tiêu là cô lập performance của database và ORM. Đưa browser rendering và network latency vào sẽ thêm các biến số không liên quan trực tiếp đến tối ưu database.

4. Dataset / Dữ liệu đo

Scenario	Dataset
Deep pagination	100,000 synthetic bookings; offset around 90,000
Bulk scheduling	20 showtimes × 120 seats = 2,400 seat snapshots
Seat repricing	120 seats across Standard, VIP and Couple types
Selective read	100 showtimes with movie, room and cinema relations

Dữ liệu synthetic nhưng tuân theo schema và quan hệ Prisma thật. Dataset dùng để so sánh hai cách triển khai trong cùng điều kiện, không đại diện cho production traffic.

5. Measurement methodology / Phương pháp đo

1. Start isolated PostgreSQL and Redis.
2. Deploy Prisma migrations.
3. Seed deterministic synthetic data and run `ANALYZE`.
4. Warm up twice.
5. Run baseline and optimized operation ten times.
6. Count Prisma query events and measure with `performance.now()`.
7. Report median and P95.

8. Reset mutation data after each iteration.
9. Use `EXPLAIN (ANALYZE, BUFFERS)` for pagination query plans.

Why median and P95 / Vì sao median và P95? Median đại diện cho lần chạy điển hình; P95 thể hiện nhóm lần chạy chậm hơn. Cách này đáng tin cậy hơn lấy lần nhanh nhất vì cache và hoạt động khác trên máy có thể ảnh hưởng kết quả.

6. Cursor pagination

Offset pagination at a deep page uses `OFFSET 90000 LIMIT 20`. Cursor pagination uses the previous row's `createdAt` and `id` with a composite index:

```
@@index([userId, createdAt, id])
WHERE ("createdAt", id) < (:cursorCreatedAt, :cursorId)
ORDER BY "createdAt" DESC, id DESC
LIMIT 21
```

Approach	Median	P95
Deep offset pagination	7.38 ms	7.87 ms
Cursor + composite index	0.26 ms	0.43 ms

- Median improvement: **96.48%**.
- P95 improvement: **94.54%**.
- Offset plan: around 90,020 rows and 1,618 buffers.
- Cursor plan: around 6 buffers for the requested range.

English interview answer

> I replaced deep offset pagination with cursor-based pagination. The cursor contains the stable sort values, `createdAt` and `id`, and the query uses a composite index on `(userId, createdAt, id)`. Offset pagination has to skip many previous rows, while the cursor query can continue from the index position. In my isolated benchmark with 100,000 synthetic bookings, median query latency at offset 90,000 decreased from 7.38 ms to 0.26 ms. This is a local synthetic benchmark, not a production SLO.

Câu trả lời tiếng Việt

> Tôi thay deep offset pagination bằng cursor-based pagination. Cursor chứa `createdAt` và `id`, còn query dùng composite index `(userId, createdAt, id)`. Offset phải bỏ qua nhiều row trước đó, còn cursor có thể tiếp tục từ vị trí trong index. Trong benchmark cô lập với 100.000 booking synthetic, median query latency tại offset 90.000 giảm từ 7.38 ms xuống 0.26 ms. Đây là benchmark local, không phải production SLO.

Follow-up: Why include id? / Vì sao thêm id?

English: `createdAt` alone may not be unique. Adding `id` creates deterministic ordering and a stable cursor boundary.

Tiếng Việt: Chỉ dùng `createdAt` có thể không unique. Thêm `id` giúp thứ tự deterministic và cursor boundary ổn định.

7. Grouped seat price updates

The original code updated every showtime seat separately. The optimized code groups seats into Standard, VIP and Couple, then uses up to three `updateMany` operations.

Approach	Business updates	Total Prisma queries	Median
One update per seat	120	122	42.07 ms
Grouped updates	3	5	3.27 ms

- Business update reduction: **120 → 3**.
- Total query reduction: **95.9%**.
- Median latency improvement: **92.23%**.

English

> The original implementation updated each showtime seat separately. Since the price is determined by seat type, I grouped the seats into Standard, VIP and Couple and used one `updateMany` per group. In a 120-seat benchmark, business update statements decreased from 120 to 3, total observed Prisma queries decreased from 122 to 5, and median duration decreased from 42.07 ms to 3.27 ms.

Tiếng Việt

> Cách cũ cập nhật từng showtime seat. Vì giá phụ thuộc loại ghế, tôi gom ghế thành Standard, VIP và Couple rồi dùng một `updateMany` cho mỗi nhóm. Trong benchmark phòng 120 ghế, số update nghiệp vụ giảm từ 120 xuống 3, tổng Prisma query giảm từ 122 xuống 5 và median duration giảm từ 42.07 ms xuống 3.27 ms.

Why not one query? / Vì sao không dùng một query?

English: The three seat types have different prices. One uniform `updateMany` cannot assign three values without a custom SQL `CASE`. Three grouped updates keep the code readable.

Tiếng Việt: Ba loại ghế có ba mức giá khác nhau. Một `updateMany` đồng nhất không thể cập nhật ba giá nếu không viết SQL `CASE`. Ba grouped updates dễ đọc và vẫn loại bỏ query theo từng ghế.

8. Batch showtime scheduling

The optimized flow uses `createManyAndReturn` for showtimes, `createMany` for seat snapshots, and reuses prefetched room/seat data.

Approach	Queries	Median	P95
Row-by-row creation	24	128.65 ms	152.35 ms
Batch creation	5	118.04 ms	138.35 ms

- Query/operation reduction: **79.17%**.
- Median latency improvement: **8.25%**.

English: The query count improved more than latency because inserting 2,400 seat snapshot rows still dominates the workload. The strongest result is lower round-trip overhead and better scalability, not a proportional latency reduction.

Tiếng Việt: Query count giảm nhiều hơn latency vì insert 2.400 seat snapshot row vẫn chiếm phần lớn workload. Kết quả mạnh nhất là giảm round-trip overhead và cải thiện scalability, không phải latency giảm theo đúng tỷ lệ query.

9. Selective Prisma projection

The optimized read replaced broad relation `include` calls with `select` projections containing only fields required by the response.

Approach	Median	Serialized data
Wide relation read	6.23 ms	110,101 bytes
Selective projection	3.12 ms	51,701 bytes

- Median latency improvement: **49.92%**.
- Serialized data reduction: **53.04%**.

English: I selected only the fields required by list responses instead of loading every Movie, Room and Cinema column. This reduced serialized data by approximately 53% and median data-access duration by approximately 50% in the local benchmark.

Tiếng Việt: Tôi chỉ lấy các field cần cho list response thay vì load toàn bộ column của Movie, Room và Cinema. Benchmark local cho thấy dữ liệu serialize giảm khoảng 53% và median data-access duration giảm khoảng 50%.

10. Other improvements / Các cải thiện khác

- Reduced seat-holding transaction scope by loading the detailed response after commit.
- Separated booking expiry cleanup from showtime and seat-map read paths.
- Processed expired bookings in bounded batches and selected only IDs for the scheduler scan.
- Added composite indexes for booking, ticket and SePay list queries.
- Added slow-query diagnostics, request query counts, service timing and scheduler timing.
- Used Redis TTL locks plus PostgreSQL transactions for seat contention and correctness.
- Retried serialization conflicts with backoff outside the transaction.

Interview wording / Cách nói:

> In addition to the four measured scenarios, I reduced the seat-holding transaction scope, moved expiry cleanup out of read-heavy seat-map requests, processed expiry in bounded batches, and added query and service timing diagnostics. I distinguish these implementation improvements from the benchmark numbers because I did not claim a production percentage for them.

11. Limitations / Giới hạn

- Data was synthetic and the benchmark ran locally in Docker.
- It measured database/data-access operations, not full browser-to-production latency.

- It did not measure production traffic, network latency, replicas, failover or autoscaling.
- Results depend on hardware, PostgreSQL cache state and dataset shape.
- The pagination result is for a deep page and should not be generalized to page one.

Do not say: “The production system is 96% faster” or “the system supports X concurrent users.”

Say instead: “In an isolated local benchmark with 100K synthetic bookings, deep-page median query latency decreased by approximately 96%.”

12. Common interview questions / Câu hỏi thường gặp

Why did you optimize pagination? / Vì sao tối ưu pagination?

English: Offset pagination becomes more expensive as the offset grows because the database still walks past previous rows. Cursor pagination gives more stable behavior for deep pages.

Tiếng Việt: Offset pagination tốn hơn khi offset lớn vì database vẫn phải đi qua các row trước đó. Cursor giúp behavior ổn định hơn ở page sâu.

What is the trade-off? / Trade-off là gì?

English: Cursor pagination is less convenient for jumping to an arbitrary page number. It requires a stable deterministic sort order and a scoped cursor.

Tiếng Việt: Cursor không tiện bằng offset khi muốn nhảy tới page bất kỳ. Nó cần sort order ổn định và cursor gắn với scope filter.

Why use Redis and PostgreSQL together?

English: Redis provides a fast TTL lock to reduce contention, while PostgreSQL remains the durable source of truth and transactionally claims the seats.

Tiếng Việt: Redis cung cấp lock TTL nhanh để giảm tranh chấp, còn PostgreSQL là source of truth và claim ghế bằng transaction.

How did you avoid a misleading benchmark?

English: I used the same schema, isolated database, fixed dataset, warm-ups and repeated runs. I reported limitations instead of presenting local results as production guarantees.

Tiếng Việt: Tôi dùng cùng schema, database cô lập, dataset cố định, warm-up và nhiều lần chạy. Tôi ghi rõ giới hạn thay vì trình bày kết quả local như production guarantee.

13. CV wording / Cách ghi vào CV

```
\resumeItem{Performance}{Optimized \textbf{PostgreSQL} and \textbf{Prisma} workloads using cursor-
based pagination, composite indexes, selective field projection, batch inserts, and grouped updates.
In an isolated local benchmark with 100K synthetic bookings, reduced deep-pagination median query
latency by \textbf{96\%}; reduced seat price updates from 120 to 3 queries; and reduced bulk showtime
database operations from 24 to 5.}
```

14. Re-run commands / Lệnh chạy lại

```
INTEGRATION_POSTGRES_PORT=55434 INTEGRATION_REDIS_PORT=56381 \
INTEGRATION_SMTP_PORT=55026 INTEGRATION_SMTP_API_PORT=58026 \
```

```
docker compose -p ticket-booking-benchmark -f be/docker-compose.integration.yml up -d --wait

DATABASE_URL='postgresql://ticket_booking_test:ticket_booking_test@127.0.0.1:55434/ticket_booking_test
?schema=public' \
REDIS_URL='redis://127.0.0.1:56381/0' NODE_ENV=test \
JWT_ACCESS_SECRET=benchmark-access JWT_REFRESH_SECRET=benchmark-refresh \
npm run prisma:deploy

DATABASE_URL='postgresql://ticket_booking_test:ticket_booking_test@127.0.0.1:55434/ticket_booking_test
?schema=public' \
BENCHMARK_BOOKINGS=100000 BENCHMARK_SHOWTIMES=100 BENCHMARK_ITERATIONS=10 \
npm run benchmark:ticket-booking -- --seed
```

Cleanup:

```
docker compose -p ticket-booking-benchmark -f be/docker-compose.integration.yml down --volumes --
remove-orphans
```

15. Final memory card / Thẻ ghi nhớ

1. Isolated Docker PostgreSQL, same Prisma migrations, synthetic data.
2. 2 warm-ups, 10 measured runs, median/P95 and Prisma query count.
3. 100K bookings, offset 90K: 7.38 ms → 0.26 ms, about 96% median improvement.
4. Seat updates: 120 business updates → 3 grouped updates.
5. Bulk scheduling: 24 queries → 5 for 20 showtimes × 120 seats.
6. Selective projection: about 53% less serialized data.
7. Always state: local synthetic benchmark, not production SLO.
